

Cloning succeeded but checkout failed due to invalid path. What is the path problem?

Asked 6 years, 4 months ago Modified 1 year, 8 months ago Viewed 52k times



I need to clone a repository that has been working with no issue on multiple linux machines, across several distributions.

28

Now it turns out that in Windows I get this problem



```
error: invalid path 'soft/android-app/app/src/main/java/com/inti/bleapp/Aux.java'
fatal: unable to checkout working tree
warning: Clone succeeded, but checkout failed.
```



And one of the biggest issues is that the cloning process takes about 15 to 20 minutes (the git has a lot of history).

And this error is thrown at the very end. I can't, for the life of me figure out why windows does nto like this path.

Can anyone give me any clues?

git

Share Improve this question Follow

asked May 5, 2020 at 13:12



[aarelovich](#)

5,692 16 73 127

- 1 just to save time on the cloning part : you can first create a bare clone on your machine, that way it won't checkout. You can then use this local bare repo as a source for debugging your path issue. You can inspect history and commits, and even use worktrees from this bare clone to inspect when the failing occurs.

– [LeGEC](#) May 5, 2020 at 13:17

Yeah, I've just figured it out. But If I can't figure out why windows does not like this path, is there anything I can do to clone the repo? – [aarelovich](#) May 5, 2020 at 13:18

- 1 I bet it's too long when you prepend the path to wherever you cloned the repository. That's a common problem when developing in Java on Windows. – [Quentin](#) May 5, 2020 at 13:19

- 1 @aarelovich: another common problem on Windows is case sensitivity : if some other file in the commit says that "soft/android-app/APP" is a file for example, it would work on linux (beacuse the file APP can live next to directory app), not on Windows. Inspect the while list of files in the branch git tries to checkout.

– [LeGEC](#) May 5, 2020 at 13:26

9 Answers

Sorted by: Highest score (default) 



How to fix it

40

If you can get someone else—on Linux for instance—to rename the problematic file and commit, that will do the job. Or just spin up a VM with Linux on it, clone, fix the name (and all users of it), commit, and send the update back.



The problem is Windows



[Windows cannot handle any file named AUX](#) . You can't name a file PRN or CON or COM1 either.



```
error: invalid path 'soft/android-app/app/src/main/java/com/inti/bleapp/Aux.java'
```

You might object: that's not a file named `AUX` . At worst, that's a file named `Aux.java` .

But to Windows, that's a file named `AUX` :

- Windows is case-insensitive, so `aux` , `Aux` , `auX` , and so on are all just way to spell `AUX` .
- Windows still thinks about files with eight-dot-three names: `FILENAME.EXT` . Sure, everything seems to use long names and you can actually have files named `foo.data.jpg` if you like. But deep in its crazy past, Windows *used to* use 8.3 names, and it insists on supporting programs that still use 8.3 names, so all files get an 8.3 name somewhere—and `Aux.java` gets an 8.3 name that starts with `AUX` , and having an extension doesn't get you around the forbidden file name.

The problem isn't just Windows

Those with macOS might gleefully point and laugh, but macOS has issues as well.

Linux folks can, and have in the past, saved files under names such as *both* `ip.h` and `IP.h` in the same directory. The idea here was to define the little-endian variant of the IP headers for TCP/IP in the lowercase `ip.h` , and define the big-endian variant in the uppercase `IP.h` . Case-folding, which both Windows and macOS do by default, gets us in trouble here.

Besides this, accented character file names—files named `schön` or `agréable` —can be built in multiple ways using UTF-8. Linux file systems can store any of the ways, and Git will just take whatever the storage method is, and put it in the committed file's name. But macOS demands one normalized form of the file name, and this causes the same kinds of problems.

What Git needs (but doesn't have)

Git needs a way to deal with this.

Git *does* have "sparse checkout". Sparse checkout allows you to define a set of files that will be checked out of some commit, and other files that won't—that will go into Git's *index* as usual but not appear in the work-tree at all.

You can use sparse checkout to check out this commit *without* extracting the file `soft/android-app/app/src/main/java/com/inti/bleapp/Aux.java` at all. Well, that's perhaps OK, but you won't have the file.

In fact, you already don't, as the final message said:

```
warning: Clone succeeded, but checkout failed.
```

When the checkout fails, all the *other* files should be available in your work-tree. It's just this one `soft/android-app/app/src/main/java/com/inti/bleapp/Aux.java` that won't. If you can get work done without it, you can simulate the sparse checkout by running:

```
git update-index --skip-worktree soft/android-app/app/src/main/java/com/inti/bleapp/Aux.java
```

What Git *should* have are some tools to deal with these files more productively. In particular, it should be possible (and is, if you get down into the depths of Git) to extract the file under *some other name*, then add the *renamed* file under the *original name* if necessary. It should be possible (and is, but not with any sensible user-facing command) to *rename* the file and commit, after which you can check out the new commit and work with the file with a name that your OS doesn't hate.

All of this is *possible* because Git actually makes new commits from what is in its index, not what is in your work-tree. There's no hard requirement that the names in the index *match* the names in your work-tree. But all of Git's user-facing commands—`git checkout`, `git restore`, and `git add` being the primary ones—*do* make this requirement today.

Share Improve this answer Follow

edited Mar 2, 2021 at 8:56

answered May 5, 2020 at 15:34



torek

504k

73

785

905

Comments

Add a comment



If you want to checkout just the files with valid paths, you can do (in Git Bash):

9

```
git clone --sparse -c core.protectNTFS=false -n <repo-URL>
git sparse-checkout add "\\!<pattern1>" "\\!<pattern2>"
git checkout <branch>
```



Patterns are relative to repo root and can use *



More info [here](#) and [here](#).

However if you actually need those files to e.g. build the project, my best bet to do this on Windows would be to use [WSL](#).

Share Improve this answer Follow

answered Aug 5, 2021 at 13:05



[Roman Orekhov](#)

432 4 7

1 Comment ^

Add a comment



[CS QGB](#) Over a year ago

error: unknown option `sparse' usage: git clone [<options>] [--] <repo> [<dir>]

▲ 0

Reply



In Linux, remove or rename files and directories with illegal Windows names. Then clone to Windows.

3

Known issues:



- characters ", *, :, <, >, ?, or \ in the name
- any of words "nul", "aux", "prn", "com1" ... "com9", or "lpt1" ... "lpt9" in the name. ex: nul.json
- name ending with "."



Share Improve this answer Follow

answered Oct 14, 2022 at 14:47



[Brian Fitzgerald](#)

956 10 19

1 Comment ^

Add a comment



[gaurav sharma](#) Over a year ago

Hi @Brian - I have a file in my source code with the name PRN.fragment.xml and git is failing to checkout this one. Why it happens for PRN words ? It is giving me error as: error: invalid path 'webapp/view/PRN.fragment.xml'

▲ 0 Reply ...



Install WSL on Windows will help. WSL helped me resolve the similar issue.

2

Share Improve this answer Follow

answered May 19, 2022 at 0:44



April

69 2 7



1 Comment ^

Add a comment



Brian Fitzgerald Over a year ago

Installing WSL on Windows is a good solution for this problem, and it's what I do on physical Windows systems.

▲ 0 Reply ...



As @Wendy said, just install de WSL on Windows, you can use this link:
<https://learn.microsoft.com/pt-br/windows/wsl/install> Btw I chose the Ubuntu.

1

In case you choose Ubuntu too, follow the next steps to install:



```
wsl --install -d Ubuntu
```



When the Ubuntu terminal shows up:

```
sudo apt update && sudo apt upgrade
```

And after you need to install git on WSL and configure, fallow the next steps:

```
sudo apt-get install git
```

Then

```
git config --global user.name "your_github_username"  
git config --global user.email "your_github_email"  
git config -l
```

You have to use a personal token to authenticate to your Git account, so follow this steps: My Account → Settings → Developer settings → Personal access tokens → Generate new token

To clone your repo use: `git clone https://<tokenhere>@github.com/<user>/<repo>.git`

To find your files from WSL in Windows explorer you can use in the repo folder this command:

```
explorer.exe .
```

Share Improve this answer Follow

edited Aug 5, 2022 at 15:11

answered Aug 5, 2022 at 14:26



Isaque Chaves

51 1 7

Comments

Add a comment



0

The checkout problem may be an issue with Windows filesystem being case insensitive, and the HEAD commit having a file whose name is a case variant of a directory (e.g : file `App` next to directory `app`).



To circumvent the "can't checkout" problem :



You can make a bare clone of your repo, and use that bare clone to inspect the commits' content.

To list the files in a commit :

From the bare clone, if the HEAD branch of the repo is `master` : you can view all the files in master using

```
git ls-tree --name-only -r master
```

Looking for possible file/directory clashes :

You can then use some scripting to turn everything to lowercase, and find possible duplicates ("bash fu" taken from [this question](#)) :

```
# if you have git-bash, and the standard linux 'tr' and 'sort' utilities :
git ls-tree --name-only -r master | tr '[:upper:]' '[:lower:]' | sort
```

```
# you can also dump the output to a file :
```

```
git ls-tree --name-only -r master > filelist.txt
# and open this file in your favorite editor/IDE for inspection
```

Since only *files* should be listed, if you see two consecutive lines where the first one is the beginning of the second one, it means a file lives next to a directory with the same name.

Share Improve this answer Follow

edited May 5, 2020 at 15:46

answered May 5, 2020 at 15:15



LeGEC

53.8k

6

72

129

Comments

Add a comment



This answer from another question seemed like the best one to me.

<https://stackoverflow.com/a/70090792/675784>

0

Run git config --global core.protectNTFS false



Then clone



Or if it's already cloned but not checked out, you may set this setting local to this repository only



Run git config core.protectNTFS false Then Run git checkout <branch_name>

Share Improve this answer Follow

answered Nov 16, 2024 at 3:01



Karl Henselin

1,047

13

17

Comments

Add a comment



As mentioned by Torek, the problem lies in the file name—in this case, AUX.java. You can use [MSYS2](#) to address this problem.

0

To install Git, run the following command in the UCRT64 terminal:



```
pacman -S git
```



After installation, use the UCRT64 terminal to run the checkout command:

Copy

 Copy

`git checkout -f HEAD`

Share Improve this answer Follow

answered Dec 17, 2024 at 20:16



Beygi

1,962

1

16

23

Comments

Add a comment



-6

If the clone succeeds, it usually means that you've been able to copy the required .git folder onto the machine. The next step requires unpacking information from that .git folder. If you don't have enough disk space, this will prevent checkout from passing.



Share Improve this answer Follow

answered May 5, 2020 at 13:28



Lloyd Moore

3,218

1

36

32



Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions

git

See similar questions with these tags.